

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Branch: CSM

Section: B

Task No.: 2

Student Name: Appala Reddy

Roll No.: A24126552089

1. TITLE

Develop an Interactive Student Result Profile Generator Using HTML5, CSS3, JavaScript Classes, and DOM Manipulation.

2. AIM

To design and implement a browser-based Student Result Profile Generator that accepts student information through an HTML form and dynamically creates a styled profile card using JavaScript and DOM manipulation, without refreshing the webpage.

3. OBJECTIVE

- To create a structured student input form using HTML5 form controls.
- To use a JavaScript class to represent student result information.
- To access webpage elements using `document.getElementById()`.
- To respond to user actions through `addEventListener("click", ...)`.
- To read and validate form values using `.value` and `.trim()`.
- To prevent profile generation when mandatory fields are incomplete.
- To dynamically create HTML elements using `document.createElement()`.
- To insert generated elements into the webpage using `appendChild()`.
- To create a reusable helper function for displaying student details.
- To apply CSS3 gradients, shadows, focus effects, hover effects, and animations for a clean profile interface.

4. THEORY

JavaScript provides classes that act as blueprints for creating objects with related properties and behaviour. In this experiment, a Student class is used to store a student's name, roll number, department, and CGPA.

The Document Object Model (DOM) represents an HTML document as a hierarchy of objects. JavaScript can access, create, modify, and insert these objects dynamically. The profile card is therefore generated at runtime from the values entered by the user rather than being permanently written in the HTML.

The main concepts used in this program are:

- `document.getElementById()` selects input, button, and container elements by their id values.
- `addEventListener("click", fn)` executes the profile-generation function when the button is pressed.
- `.value.trim()` reads the current input and removes unnecessary leading or trailing spaces.
- if conditions validate that all required fields contain values before continuing.
- `new Student(...)` creates a Student object from the entered information.
- `document.createElement()` creates new HTML elements dynamically.
- `textContent` assigns safe visible text to the generated elements.
- `appendChild()` attaches dynamically created elements to the DOM.
- `innerHTML = ""` removes an earlier generated profile before displaying the latest one.
- `:focus` and `:hover` provide visual feedback while the user interacts with the form.
- `@keyframes` creates a smooth entrance animation for the generated profile card.

Basic flow: User enters student details → button is clicked → values are read and trimmed → validation is performed → Student object is created → profile elements are generated using DOM methods → profile card is displayed on the same page.

5. ALGORITHM / PROCEDURE

1. Create three files: index.html, style.css, and script.js, and connect the CSS and JavaScript files to the HTML page.
2. Design a centered input card with fields for Student Name, Roll Number, Department, and CGPA.
3. Add a Generate Student Profile button below the input fields.
4. Create an empty section with an id such as profileContainer for the dynamically generated result.
5. Define a Student class in JavaScript with properties name, rollNumber, department, and cgpa.
6. Select the four input elements, button, and profile container using document.getElementById().
7. Attach a click event listener to the Generate Student Profile button.
8. Read the values from all input fields using .value.trim().
9. Check whether any field is empty; if so, display an alert and stop execution.
10. Create a Student object using the validated values.
11. Clear any previously generated profile card.
12. Create a profile card and a header section dynamically using document.createElement().
13. Create a reusable helper function to generate each student-detail row.
14. Add the Name, Roll Number, Department, and CGPA rows to the profile details section.
15. Append the completed header and details to the profile card and then append the card to the container.
16. Save all files, open index.html in a browser, and test valid, invalid, repeated, and interactive inputs.

6. PROGRAM

index.html, style.css and script.js

```
index.html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Student Result Profile Generator</title>
<link rel="stylesheet" href="style.css">
</head>
<body>

<div class="container">
<header>
<h1>☐ Student Result Profile</h1>
<p>Enter academic details and generate a profile dynamically</p>
</header>

<section class="form-card">
<h2>Student Input Form</h2>

<div class="input-group">
<label for="name">Name</label>
<input type="text" id="name" placeholder="Enter student name">
</div>

<div class="input-group">
<label for="roll">Roll No</label>
<input type="text" id="roll" placeholder="Enter roll number">
</div>

<div class="input-group">
<label for="department">Department</label>
<input type="text" id="department" placeholder="Enter department">
</div>

<div class="input-group">
<label for="cgpa">CGPA</label>
<input type="number" id="cgpa" step="0.01" min="0" max="10"
placeholder="Enter CGPA">
</div>

<button id="generateBtn">Generate Profile</button>
</section>

<section id="profileContainer"></section>
</div>

<script src="script.js"></script>
</body>
</html>

style.css
* {
margin: 0;
padding: 0;
box-sizing: border-box;
}

body {
font-family: Arial, sans-serif;
min-height: 100vh;
background: linear-gradient(135deg, #f4f7f8, #dce7ee);
display: flex;
justify-content: center;
align-items: flex-start;
padding: 38px 20px;
}

.container {
```

```
width: 100%;
max-width: 700px;
}

header {
text-align: center;
margin-bottom: 24px;
}

header h1 {
font-size: 30px;
margin-bottom: 8px;
}

header p {
font-size: 15px;
color: #555;
}

.form-card {
background: white;
padding: 30px;
border-radius: 14px;
box-shadow: 0 8px 24px rgba(0,0,0,0.14);
margin-bottom: 22px;
}

.form-card h2 {
margin-bottom: 20px;
}

.input-group {
margin-bottom: 15px;
}

.input-group label {
display: block;
font-weight: bold;
margin-bottom: 6px;
}

.input-group input {
width: 100%;
padding: 11px;
border: 1px solid #ccc;
border-radius: 6px;
font-size: 15px;
}

.input-group input:focus {
outline: none;
border-color: #2388e8;
box-shadow: 0 0 5px rgba(35,136,232,0.25);
}

button {
width: 100%;
padding: 13px;
border: none;
border-radius: 6px;
background: #147cf5;
color: white;
font-size: 16px;
font-weight: bold;
cursor: pointer;
transition: 0.3s;
}

button:hover {
background: #0e63c5;
transform: translateY(-2px);
}
```

```

.profile-card {
background: white;
border-radius: 14px;
overflow: hidden;
box-shadow: 0 8px 24px rgba(0,0,0,0.14);
animation: revealProfile 0.45s ease;
}

.profile-header {
background: linear-gradient(135deg, #1f6fb2, #36a6d8);
color: white;
text-align: center;
padding: 24px;
}

.profile-header .icon {
font-size: 40px;
margin-bottom: 7px;
}

.profile-header h2 {
font-size: 23px;
}

.profile-details {
padding: 22px;
}

.detail {
display: flex;
justify-content: space-between;
padding: 13px 4px;
border-bottom: 1px solid #eee;
}

.detail:last-child {
border-bottom: none;
}

.detail .label {
font-weight: bold;
color: #555;
}

.detail .value {
color: #222;
text-align: right;
}

.cgpa {
font-weight: bold;
color: #147cf5 !important;
}

@keyframes revealProfile {
from {
opacity: 0;
transform: translateY(18px);
}
to {
opacity: 1;
transform: translateY(0);
}
}

@media (max-width: 600px) {
body {
padding: 20px 12px;
}
header h1 {
font-size: 25px;
}

```

```

}
.form-card {
padding: 20px;
}
.detail {
flex-direction: column;
gap: 5px;
}
.detail .value {
text-align: left;
}
}

script.js
class Student {
constructor(name, rollNumber, department, cgpa) {
this.name = name;
this.rollNumber = rollNumber;
this.department = department;
this.cgpa = cgpa;
}
}

const nameInput = document.getElementById("name");
const rollInput = document.getElementById("roll");
const departmentInput = document.getElementById("department");
const cgpaInput = document.getElementById("cgpa");
const generateButton = document.getElementById("generateBtn");
const profileContainer = document.getElementById("profileContainer");

generateButton.addEventListener("click", function () {
const name = nameInput.value.trim();
const rollNumber = rollInput.value.trim();
const department = departmentInput.value.trim();
const cgpa = cgpaInput.value.trim();

if (name === "" || rollNumber === "" ||
department === "" || cgpa === "") {
alert("Please enter all student details.");
return;
}

const student = new Student(
name,
rollNumber,
department,
cgpa
);

profileContainer.innerHTML = "";

const profileCard = document.createElement("div");
profileCard.className = "profile-card";

const profileHeader = document.createElement("div");
profileHeader.className = "profile-header";

const icon = document.createElement("div");
icon.className = "icon";
icon.textContent = "□";

const heading = document.createElement("h2");
heading.textContent = "Student Result Profile";

profileHeader.appendChild(icon);
profileHeader.appendChild(heading);

const profileDetails = document.createElement("div");
profileDetails.className = "profile-details";

function createDetail(labelText, valueText, extraClass = "") {
const detail = document.createElement("div");

```

```
detail.className = "detail";

const label = document.createElement("span");
label.className = "label";
label.textContent = labelText;

const value = document.createElement("span");
value.className = "value " + extraClass;
value.textContent = valueText;

detail.appendChild(label);
detail.appendChild(value);
return detail;
}

profileDetails.appendChild(createDetail("Name", student.name));
profileDetails.appendChild(createDetail("Roll No", student.rollNumber));
profileDetails.appendChild(createDetail("Department", student.department));
profileDetails.appendChild(createDetail("CGPA", student.cgpa, "cgpa"));

profileCard.appendChild(profileHeader);
profileCard.appendChild(profileDetails);
profileContainer.appendChild(profileCard);
});
```

7. CODE EXPLANATION

Code / Concept	Description
class Student { constructor(...) }	Defines a reusable blueprint for storing student name, roll number, department, and CGPA.
new Student(...)	Creates a Student object using the values entered by the user.
document.getElementById()	Selects the required form fields, button, and output container.
addEventListener("click", fn)	Runs the profile-generation logic whenever the Generate Profile button is clicked.
.value.trim()	Reads the current input and removes extra whitespace from the beginning and end.
if (... === "")	Checks whether any required field is empty before creating the profile.
alert(...); return;	Informs the user about missing details and stops the function from continuing.
profileContainer.innerHTML = ""	Clears an older result so that only the latest profile is displayed.
document.createElement()	Creates new HTML elements dynamically in JavaScript.
textContent	Places text safely inside the newly created elements.
appendChild()	Attaches each generated element to its parent in the DOM.
createDetail(label, value, extraClass)	Reusable helper that builds one student information row and reduces repeated code.
:focus / :hover	Provides visual feedback when an input receives focus or the button is hovered.
@keyframes revealProfile	Animates the generated profile card as it appears on the page.

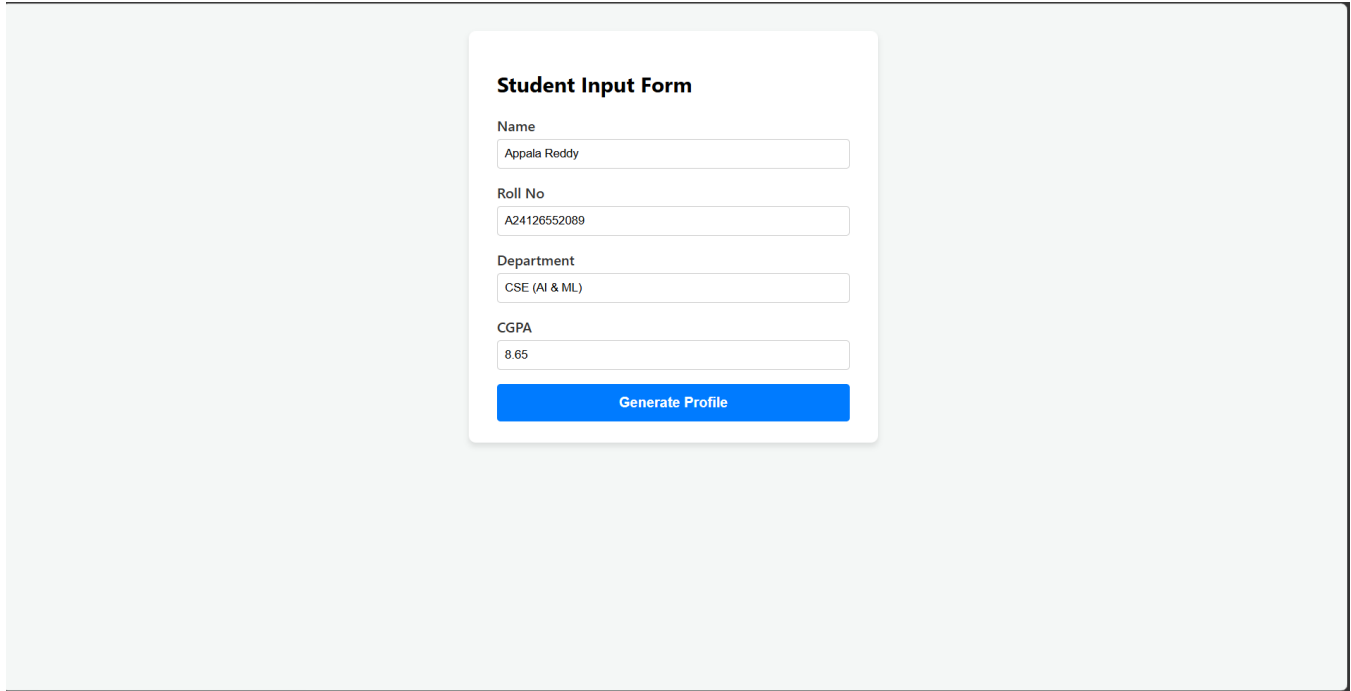
8. TEST CASES AND EXECUTION DATA

The revised webpage was tested in a browser using valid and invalid student inputs. The following cases verify form validation, profile generation, repeated generation, and interface behaviour.

Test Case	Action / Input	Expected Result	Actual Result	Status
TC-01	Open index.html in the browser	Student Input Form appears with four labelled fields and a Generate Profile button	Form displayed correctly	Pass
TC-02	Click Generate Profile with all fields empty	Validation alert appears and no profile is generated	Alert displayed and profile was not created	Pass
TC-03	Enter Appala Reddy, A24126552089, CSE (AI & ML), and 8.65	A profile card is generated with the entered values without page reload	Profile card displayed with correct values	Pass
TC-04	Enter CGPA greater than 10	The number field prevents values outside the defined maximum range	Maximum range validation worked	Pass
TC-05	Generate a profile, then change the details and click the button again	The old profile is cleared and replaced by the latest profile	Previous profile replaced successfully	Pass
TC-06	Leave Department blank and click Generate Profile	Validation alert appears and no new profile is created	Alert displayed as expected	Pass
TC-07	Click inside a text input	Input border and focus glow change	Focus styling displayed correctly	Pass
TC-08	Hover over Generate Profile	Button darkens and moves slightly upward	Hover animation displayed correctly	Pass
TC-09	Enter a name with extra spaces	trim() removes unnecessary leading and trailing spaces	Input was cleaned before profile creation	Pass
TC-10	Resize the browser to a narrow width	Layout remains usable and detail rows stack vertically	Responsive layout worked correctly	Pass

9. OUTPUT

Output: Student Input Form and Generated Student Profile



The screenshot displays a web browser window with a light gray background. Centered on the page is a white rectangular form titled "Student Input Form" in bold black text. The form contains four input fields, each with a label above it: "Name" (containing "Appala Reddy"), "Roll No" (containing "A24126552089"), "Department" (containing "CSE (AI & ML)"), and "CGPA" (containing "8.65"). Below these fields is a prominent blue button with the text "Generate Profile" in white. The form is set against a light gray background within the browser window.

The browser output above shows the completed Student Input Form with the revised student name and roll number. The same page is designed to generate the corresponding profile dynamically when the Generate Profile button is clicked.

10. RESULT

Thus, an interactive Student Result Profile Generator was successfully developed using HTML, CSS, JavaScript classes, and DOM manipulation. The application accepts student details, validates the inputs, creates a Student object, and dynamically generates a styled profile without reloading the webpage. The revised implementation also provides focus, hover, animation, and responsive-layout behaviour. The output shown above was retained as the experiment output.